MACHINE LEARNING I

RAPPORT :

Projet 1 : PREDICTIVE MAINTENANCE : HYDRAULIC VALVE CONDITION CLASSIFICATION

MEMBRES :

- **TSAPI KOUDJOU Alex Cabrel**
- **NZIMEU KENMOE Joyce Ingrid**

Encadrant : Mr Ahmad Tay
Année universitaire 2026-2027

OBJECTIVE :

Dans un contexte industriel, être capable de repérer en avance les signes de dégradation d'une machine industrielle à un grand impact sur la productivité.

Dans ce projet, nous étudions le fonctionnement d'une valve hydraulique à partir des mesures de pression et de débit volumique.

Notre objectif est de construire un model prédictif capables de signaler aux opérateurs si la valve fonctionne de manière optimale ou non pendant un cycle. Pour cela, nous allons utiliser 3 fichiers :

- PS2.txt contient les mesures de pression enregistrées à une fréquence de 100 Hz.

- FS1.txt contient les mesures de débit, en litres par minute, enregistrées à une fréquence de 10 Hz.

- Profile.txt contient les conditions de fonctionnement du système, dont l'état de la valve.

Le rapport détaille le traitement des données, la construction et la sélection des caractéristiques, la comparaison des modèles et leur optimisation. Il présente également l'application développée pour consulter une prédiction par numéro de cycle.

SOMMAIRE :

1. Introduction et problématique

2. Présentation des données et protocole expérimental

3. Analyse exploratoire et qualité des données

4. Construction des caractéristiques

5. Analyse et sélection des caractéristiques

6. Préparation et choix des modèles

7. Comparaison des modèles de référence

8. Optimisation et généralisation

9. Évaluation sur le test final

10. Interprétation et limites

11. Application et déploiement

12. Conclusion générale

Références

Annexes

1. INTRODUCTION ET PROBLEMATIQUE

a. CONTEXTE ET OBJECTIF

Ce projet vise à mettre en place un système de maintenance prédictive pour les machines de l'usine et à analyser les causes qui font que la condition de la valve d'un cycle de production est non optimale parfois.

Nous devons construire un modele de machine Learning capable de reconnaître l'état de la valve à partir des mesures d'un cycle complet. Le modèle doit distinguer deux classes : un fonctionnement optimal et un fonctionnement non optimal.

b. DEMARCHE GENERALE

Comme tout projet de data science nous allons commencer par le processus data engineering ie, ETL (Extract transform and load). Nous avons commencer par charger les donnees et vérifier leur qualité. Ensuite nous leur avons transformer en pandas dataframe pour pourvoir les manipuler facilement. Ensuite, nous avons étudier les signaux et avons construite des caractéristiques pour résumer chaque cycle. Après avoir analysé ces caractéristiques, nous en sélectionnons une partie pour entraîner et comparer les modèles. Le test final doit permettre de vérifier les performances sur les cycles réservés.

2. PRESENTATION DES DONNEES ET PROTOCOLE EXPERIMENTAL

a. SOURCE ET FICHIERS UTILISES

Comme indique dans l'énoncé, nous avons utilisé les trois fichiers dans l'énoncé. Les données des fichiers était séparé par la tabulation. Nous avons utilisé la fonction read_csv() de pandas.

Fichier	Information utilisée	Échantillonnage	Taille
PS2	Pression en bar	100 Hz	2 205 × 6 000
FS1	Débit en L/min	10 Hz	2 205 × 600
Profile	Condition de valve dans la deuxième colonne	Une étiquette par cycle	2 205 étiquettes

La condition de la valve est extraite de la deuxième colonne du dataframe profile et stocke dans le dataframe valve_condition.

Chaque ligne représente un cycle de fonctionnement de 60 secondes. Les colonnes de PS2 et FS1 correspondent aux mesures prises pendant ce cycle. Les fichiers suivent le même ordre de cycles selon la documentation.

```
Nombre de cycles dans FS1: 2205
Nombre de cycles dans PS2: 2205
Nombre de target values: 2205

Meme nombre de cycles: True
```

Nous vérifions aussi qu'ils contiennent chacun 2 205 lignes. Les mesures des deux capteurs et la condition de valve d'une même ligne peuvent ainsi être associées. Ainsi on peut faire une correspondance direct entre les données de pression, débit et fonctionnement de la valve.

b. DEFINITION DE LA CIBLE

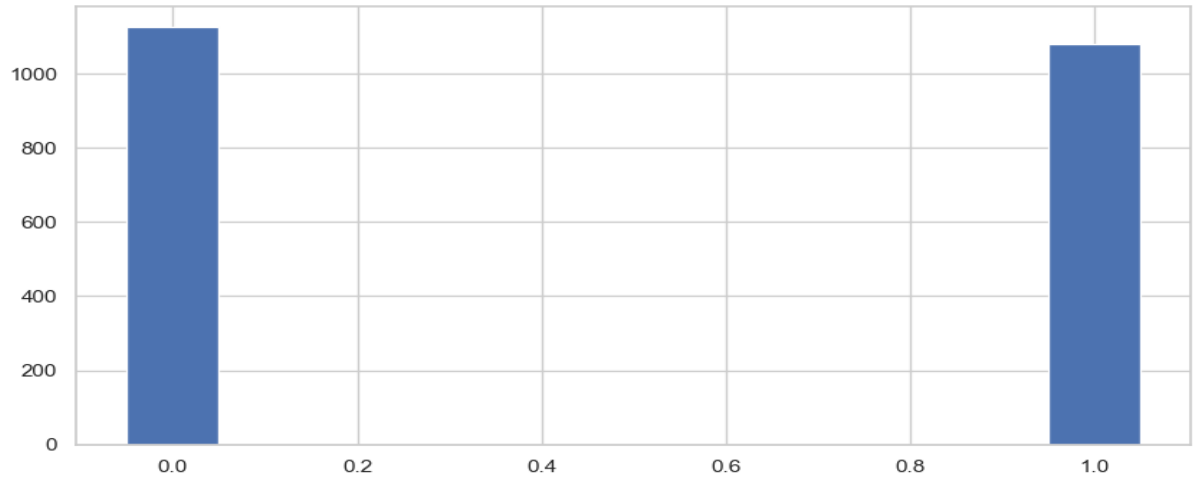
Nous examinons les valeurs présentes dans valve_condition. Nous trouvons quatre conditions : 100, 90, 80 et 73 %. Notre objectif étant une classification binaire, nous regroupons ces conditions en deux classes. La valeur 100 correspond à la classe optimale ; toutes les autres valeurs correspondent à la classe non optimale.

État initial	Nombre de cycles	Classe binaire
100 %	1 125	0 Optimal
90 %	360	1 Non optimal
80 %	360	1 Non optimal
73 %	360	1 Non optimal

c. REPARTITION DES CLASSES

Apres avoir encoder notre target, nous avons étudier sa distribution dans notre dataset pour savoir les proportions des modes de fonctionnement de la valve

Classe	Effectif	Proportion
Optimal 0	1 125	51,02 %
Non optimal 1	1 080	48,98 %



Les deux classes sont proches : 51,02 % des cycles sont optimaux et 48,98 % sont non optimaux. Nous ne constatons donc pas de déséquilibre significatif dans l'ensemble des données.

d. ORGANISATION DE L'EVALUATION

Conformément à l'énoncé, nous utilisons les 2 000 premiers cycles pour développer le modèle et réservons les 205 derniers au test final.

```
#Separer les 2000 premières lignes
X_main = X.iloc[:2000].copy()
y_main = y.iloc[:2000].copy()

#Hold-out
X_holdout = X.iloc[2000:].copy()
y_holdout = y.iloc[2000:].copy()

print("X_main:", X_main.shape)
print("X_holdout:", X_holdout.shape)
```

[32] ✓ 0.0s

```
X_main: (2000, 21)
X_holdout: (205, 21)
```

Les dataframes X_main et X_holdout désignent respectivement la validation interne et le test final. Nous employons ici les noms fonctionnels pour éviter la confusion. Les décisions de sélection et de standardisation doivent être apprises sans utiliser l'ensemble sur lequel elles sont évaluées.

3. ANALYSE EXPLORATOIRE ET QUALITE DES DONNEES

a. CONTROLES DE COHERENCE

Nous vérifions ensuite les dimensions et les types. FS1 contient 600 mesures par cycle et PS2 en contient 6 000.

Contrôle	PS2	FS1	Code
Valeurs manquantes	0	0	
Valeurs infinies	0	0	
Lignes identiques	0	0	

	Dataset	Missing Values
0	PS2	0
1	FS1	0
2	Valve Condition	0

```
Infinite values in PS2: 0
Infinite values in FS1: 0
```

```
Duplicate PS2 cycles: 0
Duplicate FS1 cycles: 0
```

Valeurs négatives	0	0	Negative PS2 values: 0 Negative FS1 values: 0

Nous ne trouvons aucune valeur manquante dans PS2, FS1 et la cible. Les contrôles ne détectent pas non plus de valeur infinie dans les deux capteurs ni de ligne identique dans chacun de ces fichiers. Aucun remplacement de valeur n'est nécessaire pour ces points.

b. STATISTIQUES DESCRIPTIVES

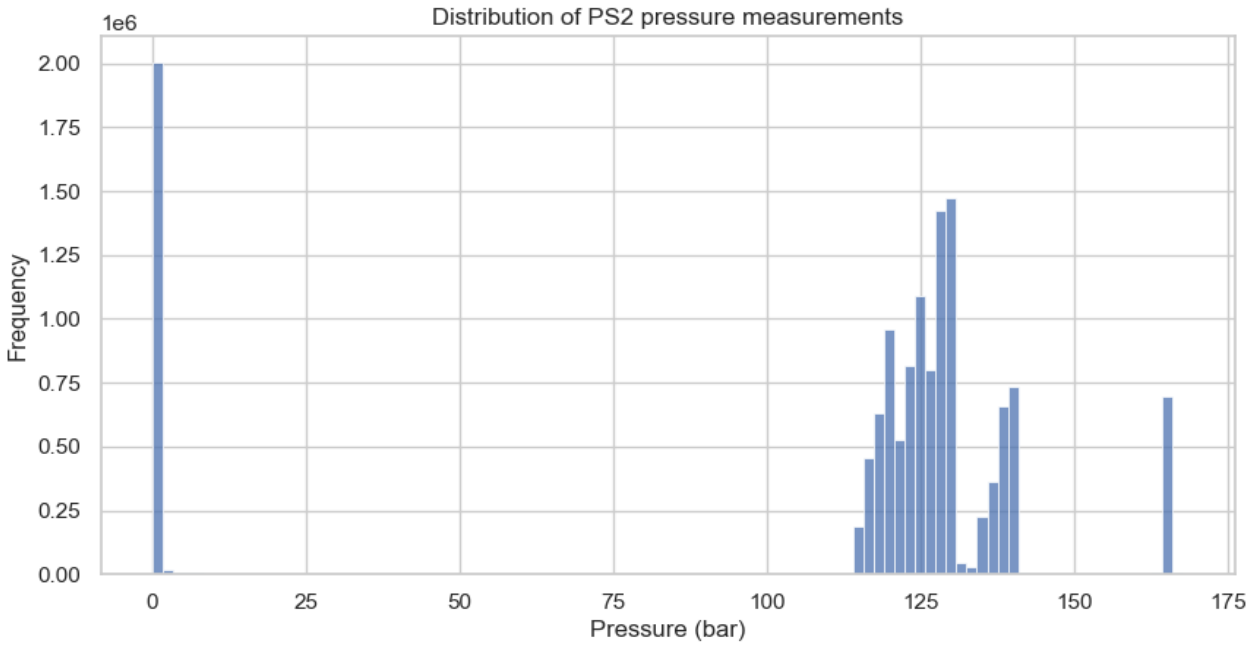
Nous examinons les niveaux de pression et de débit ainsi que leur dispersion.

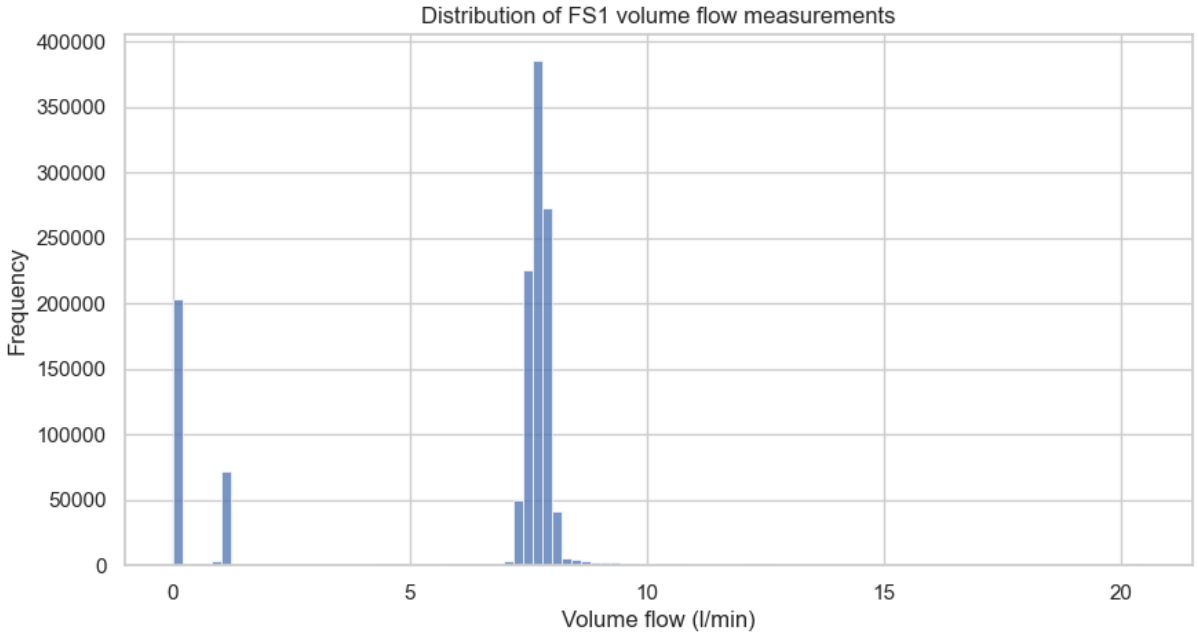
Indicateur	PS2 en bar	FS1 en L/min
Moyenne	109,38	6,20
Écart type	48,10	3,21
Minimum	0	0
Médiane	125,70	7,64
Maximum	167,77	20,479

Le tableau regroupe toutes les mesures de chaque capteur. La pression moyenne est d'environ 109,38 bar et le débit moyen de 6,20 L/min. Ces valeurs décrivent l'ensemble des mesures, mais ne montrent pas leur évolution au cours d'un cycle.

c. VALEURS EXTREMES ET NETTOYAGE

Nous allons procéder par tracer les histogrammes des valeurs de ps2 et de fs1 pour pouvoir voir leurs distributions et facilement identifier les valeurs extrêmes et des éventuelles outliers.





La distribution de pression et de débit volumique est irrégulière. Nous observons des valeurs proches de zéro et de nombreuses mesures plus élevées. Les différentes phases de fonctionnement pendant un cycle peuvent contribuer à cette forme (Phase de chargement et de déchargement). Nous conservons les valeurs extrêmes à ce stade, car les graphiques ne permettent pas de les identifier comme des erreurs de mesure.

d. OBSERVATION DES SIGNAUX

Nous traçons un cycle pour chacune des quatre conditions de valve afin de visualiser les mesures. Les courbes permettent d'observer les changements de pression et de débit pendant le fonctionnement. Ces exemples illustrent les signaux, mais un seul cycle par condition ne suffit pas à décrire toute la classe.

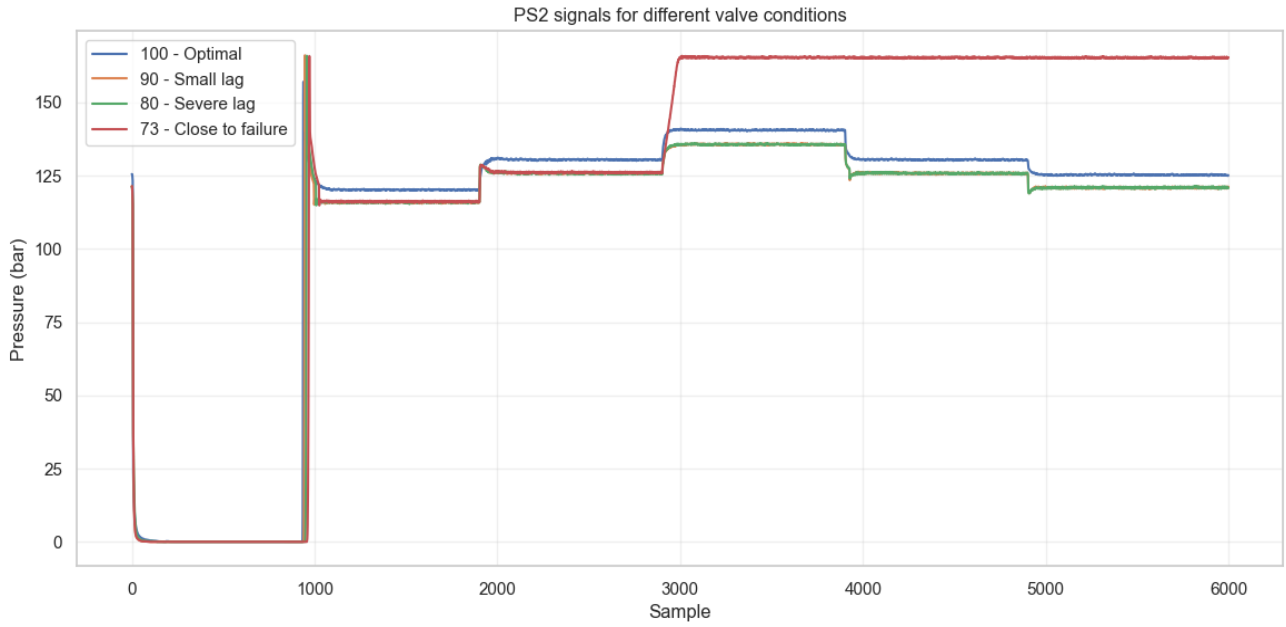


Figure 1 Signaux PS2 pour un cycle de chaque condition de valve

Et nous traçons aussi pour fs1.

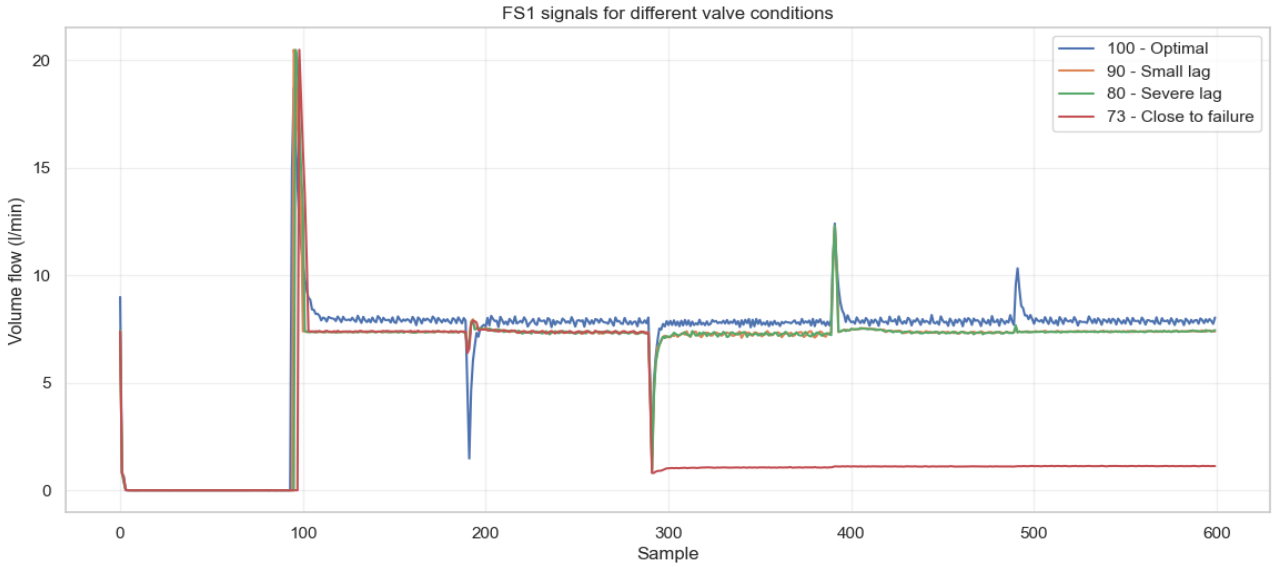
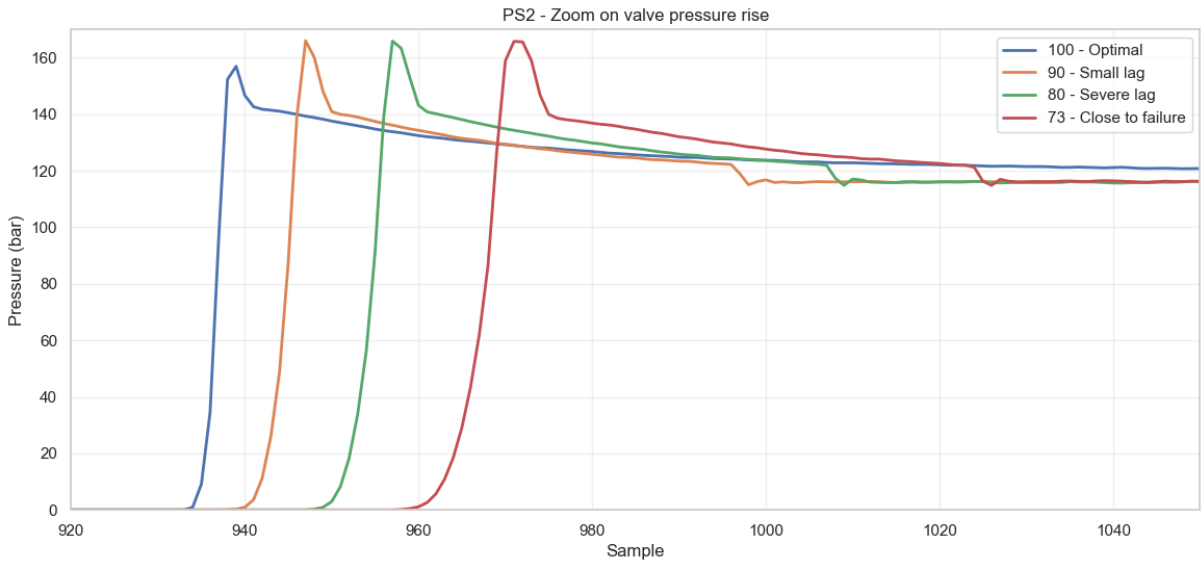


Figure 2 Signaux FS1 pour un cycle de chaque condition de valve

Les graphiques enregistrés utilisent le numéro d'échantillon. Pour les comparer sur une même durée, l'axe temporel se calcule par t = numéro d'échantillon / fréquence. Un échantillon PS2 représente 0,01 seconde, contre 0,1 seconde pour FS1.

Ainsi, nous pouvons observer l'évolution temporelle des données des capteurs pour différents mode de fonctionnement. Ainsi nous pouvons observer comment les modes de fonctionnement influence la pression et le débit volumique dans notre valve.



On constate facilement que dépendant du comportement de la valve, les différentes pressions montent et chute différemment. Ainsi, nous pouvons facilement identifier le comportent de la valve par rapport à son comportement proche ces zones critiques.

Ces informations nous ont aidé à mieux comprendre les données.

e. BILAN DE L'EXPLORATION

Les vérifications réalisées ne signalent pas de valeur manquante ou infinie dans les capteurs. Les courbes et les distributions montrent cependant que les mesures varient pendant le cycle. Nous allons donc traiter les données comme des signaux et calculer plusieurs caractéristiques pour résumer ces variations.

4.CONSTRUCTION DES CARACTERISTIQUES

a. EXTRACTION PAR CYCLE

Les données PS2 et FS1 sont constituées de séries temporelles enregistrées pendant chaque cycle de fonctionnement de 60 secondes. Le capteur de pression PS2 est échantillonné à 100 Hz, ce qui correspond à 6 000 mesures par cycle, tandis que le capteur de débit FS1 est échantillonné à 10 Hz, soit 600 mesures par cycle.

L'utilisation directe de ces milliers de mesures comme variables d'entrée aurait fortement augmenté la dimension du problème. Un travail de *feature engineering* a donc été réalisé afin de transformer chaque cycle en un ensemble réduit de caractéristiques décrivant à la fois le niveau global des signaux, leurs variations, leur contenu fréquentiel et certains événements caractéristiques du cycle.

L'extraction produit au total 44 caractéristiques par cycle, dont 23 pour PS2 et 21 pour FS1.

Famille	Caractéristiques	Information résumée
Statistiques globales	Moyenne, médiane, écart-type, minimum, maximum, étendue	Niveau général, dispersion et valeurs extrêmes
Variation temporelle	Variation absolue moyenne	Intensité moyenne des changements entre mesures successives
Énergie	Moyenne des valeurs au carré	Importance globale de l'amplitude du signal
Fréquentielles	Fréquence dominante, entropie spectrale	Organisation du signal dans le domaine fréquentiel
Pics	Nombre de pics	Nombre d'événements locaux importants
Fenêtres glissantes	Écart-type de la moyenne glissante, maximum de l'écart-type glissant	Variabilité locale sur une fenêtre d'une seconde

Transitions	Temps de franchissement d'un seuil, instant du premier pic, chute après le pic	Dynamique de réaction du signal lors du premier événement
Plateau	Moyenne et écart-type du plateau	Niveau et stabilité après le premier événement
Changements au cours du cycle	Profondeur du creux vers 20 s, variation autour de 30 s et 39 s	Évolution du signal pendant différentes phases du cycle
PS2 spécifiques	Durée entre franchissement du seuil et pic, chute autour de 49 s	Dynamique supplémentaire du signal de pression

Les seuils utilisés pour détecter le premier franchissement sont de 100 bar pour PS2 et de 5 l/min pour FS1. L'analyse est effectuée indépendamment pour chaque cycle et sans utiliser la classe cible.

Cette transformation permet donc de passer de plusieurs milliers de mesures brutes à 44 variables synthétiques par cycle, mieux adaptées aux algorithmes de classification utilisés ensuite.

b. STATISTIQUES ET VARIATIONS TEMPORELLES

Une première famille de caractéristiques décrit le comportement général des signaux sur l'ensemble du cycle.

Pour chaque capteur, la moyenne, la médiane, l'écart-type, le minimum, le maximum et l'étendue sont calculés. Ces mesures résument respectivement le niveau moyen du signal, sa valeur centrale, sa dispersion et ses valeurs extrêmes.

Le notebook ajoute également plusieurs caractéristiques directement liées aux transitions observées sur les courbes. Une fenêtre située entre 8 et 12 secondes est notamment utilisée pour identifier le premier événement important du signal.

Le temps de franchissement du seuil correspond au premier instant où PS2 dépasse 100 bar ou FS1 dépasse 5 l/min. Le temps du premier pic correspond à l'instant du maximum dans cette même fenêtre.

Ces variables sont particulièrement intéressantes car l'analyse visuelle des cycles a montré que la montée des signaux se produit avec un décalage différent selon l'état de la valve.

c. FREQUENCES ET ENERGIE

Afin de compléter les statistiques temporelles, des informations sur l'énergie et le contenu fréquentiel des signaux sont également extraites.

Cette mesure permet de caractériser l'amplitude globale du signal sur l'ensemble du cycle.

Une transformée de Fourier rapide, ou FFT, est ensuite appliquée au signal centré. La composante de fréquence nulle est exclue afin de ne pas prendre en compte la moyenne du signal.

Deux caractéristiques sont alors calculées :

- la fréquence dominante, correspondant à la fréquence possédant la puissance la plus élevée ;
- l'entropie spectrale, qui mesure la manière dont l'énergie est distribuée entre les différentes fréquences.

L'entropie spectrale est faible lorsque l'énergie est concentrée autour de quelques fréquences et plus élevée lorsqu'elle est répartie sur un plus grand nombre de composantes.

Cette approche permet d'ajouter une information complémentaire aux seules statistiques calculées dans le domaine temporel.

d. PICS ET VERIFICATION

Les pics présents dans chaque cycle sont détectés à l'aide de find_peaks. Un critère de proéminence égal à 0,5 fois l'écart-type du signal est utilisé afin de ne conserver que les variations suffisamment marquées.

Le nombre de pics constitue une caractéristique supplémentaire permettant de décrire la présence d'événements locaux dans le signal.

Des statistiques calculées sur une fenêtre glissante d'une seconde sont également ajoutées. La fenêtre correspond donc à :

- 100 mesures pour PS2 ;
- 10 mesures pour FS1.

Deux caractéristiques sont retenues :

- l'écart-type de la moyenne glissante ;
- le maximum de l'écart-type glissant.

Ces variables permettent de caractériser les variations locales qui peuvent être masquées par les statistiques calculées sur l'ensemble du cycle.

D'autres caractéristiques décrivent ensuite plusieurs phases précises du cycle : le niveau du plateau entre 12 et 18 secondes, un creux autour de 20 secondes et des changements de niveau autour de 30 et 39 secondes.

Après extraction, la matrice finale contient 2 205 cycles et 44 caractéristiques.

5. ANALYSE ET SELECTION DES CARACTERISTIQUES

Avant d'effectuer la sélection des variables, les données sont séparées afin d'éviter toute fuite d'information.

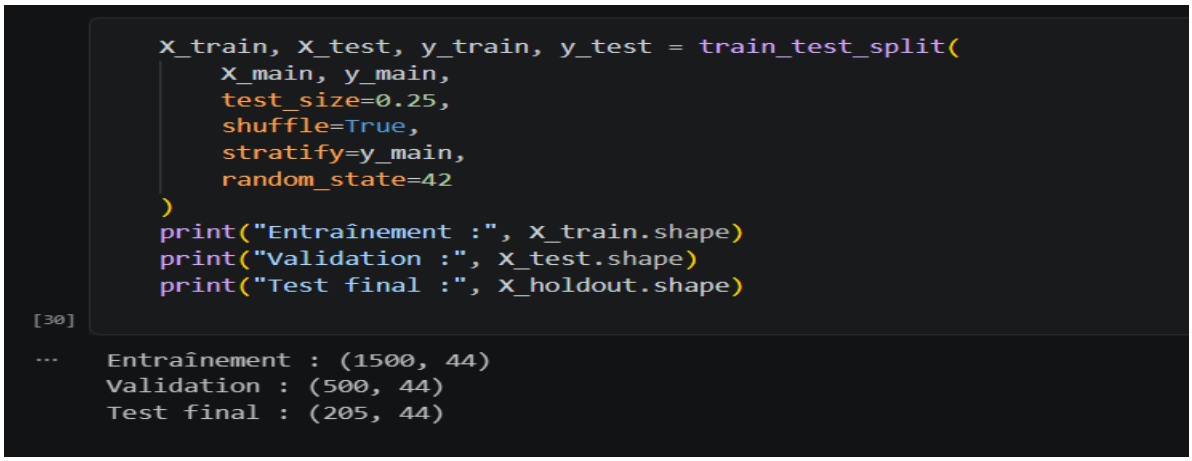
Conformément à l'énoncé, les 205 derniers cycles sont isolés dès le départ et constituent le jeu de test final (*holdout*). Ils ne sont pas utilisés lors de l'analyse et de la sélection des caractéristiques.

Les 2 000 premiers cycles sont ensuite divisés en :

- 1 500 cycles d'entraînement ;
- 500 cycles de validation.

La séparation est réalisée avec stratify=y_main afin de conserver une proportion comparable de valves optimales et non optimales dans les deux ensembles.

Toutes les décisions de sélection des caractéristiques sont effectuées uniquement à partir des 1 500 observations d'entraînement.



Ainsi, on évite data leakage.

a. VARIABLES CONSTANTES ET REDONDANCES DIRECTES

La première étape consiste à identifier les variables qui ne varient pas dans les données d'entraînement.

Deux caractéristiques présentent une valeur constante :

- PS2_min ;

- PS2_dominant_frequency.

Ces deux variables ont donc été supprimées, car une caractéristique constante ne peut contribuer à distinguer les deux classes.

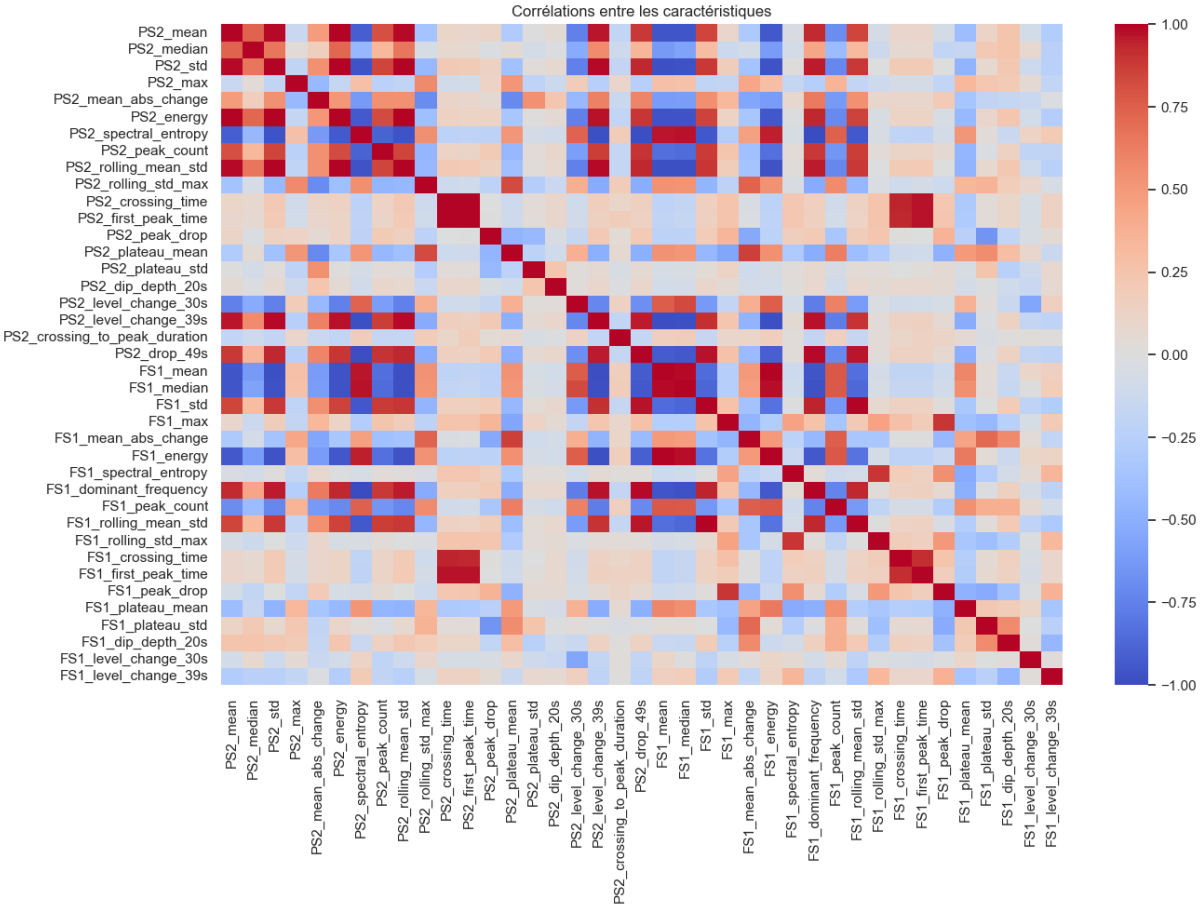
Le nombre de variables passe alors de **44 à 42**.

Trois autres variables sont ensuite supprimées en raison de redondances directes :

- PS2_range ;
- FS1_min ;
- FS1_range.

b. CORRELATIONS ENTRE CARACTERISTIQUES

Une matrice de corrélation est ensuite calculée uniquement sur les données d'entraînement.



Cette analyse montre que plusieurs caractéristiques évoluent de manière très similaire. C'est notamment le cas de certaines variables issues de PS2, telles que la moyenne, l'écart-type, l'énergie ou les statistiques glissantes.

Des relations similaires sont observées pour certaines caractéristiques de FS1.

Afin d'identifier les redondances les plus fortes, les paires de variables dont la valeur absolue de la corrélation est supérieure ou égale à **0,95** sont isolées.

Pour chaque paire, trois informations sont comparées :

- La corrélation entre les deux caractéristiques ;
- La corrélation de la première caractéristique avec la cible ;
- La corrélation de la seconde caractéristique avec la cible.

Cette démarche permet de réduire la redondance tout en conservant, lorsque cela est possible, la variable la plus liée à l'état de la valve.

c. RELATIONS AVEC LA CIBLE

La corrélation entre chaque caractéristique et la cible binaire est calculée à partir des données d'entraînement.

L'analyse montre notamment que les caractéristiques associées aux **instants de franchissement des seuils et aux pics** présentent des relations importantes avec l'état de la valve.

Cela confirme l'observation faite lors de l'analyse visuelle des signaux : lorsque la valve se dégrade, certaines transitions se produisent plus tard.

Ainsi, les variables temporelles décrivant le retard de réaction des signaux apportent une information particulièrement utile pour différencier les cycles optimaux des cycles non optimaux.

Certaines caractéristiques liées au débit FS1 montrent également une relation avec l'état de la valve.

Cependant, une caractéristique présentant une faible corrélation linéaire avec la cible n'est pas automatiquement supprimée. Elle peut rester utile à des modèles capables d'exploiter des relations non linéaires.

d. CHOIX DES CARACTERISTIQUES

À partir de l'analyse des fortes corrélations, 11 caractéristiques supplémentaires sont retirées : PS2_first_peak_time, FS1_first_peak_time, PS2_std, PS2_energy, PS2_rolling_mean_std, PS2_spectral_entropy, PS2_level_change_39s, FS1_mean, FS1_median, FS1_rolling_mean_std, PS2_drop_49s

Lorsqu'une paire de variables est fortement corrélée, le choix privilégie celle qui présente la relation la plus importante avec la cible.

Cette sélection permet de réduire la dimension du problème tout en conservant différentes familles d'information : statistiques globales, dynamique temporelle, comportement fréquentiel, pics, plateaux et transitions du cycle.

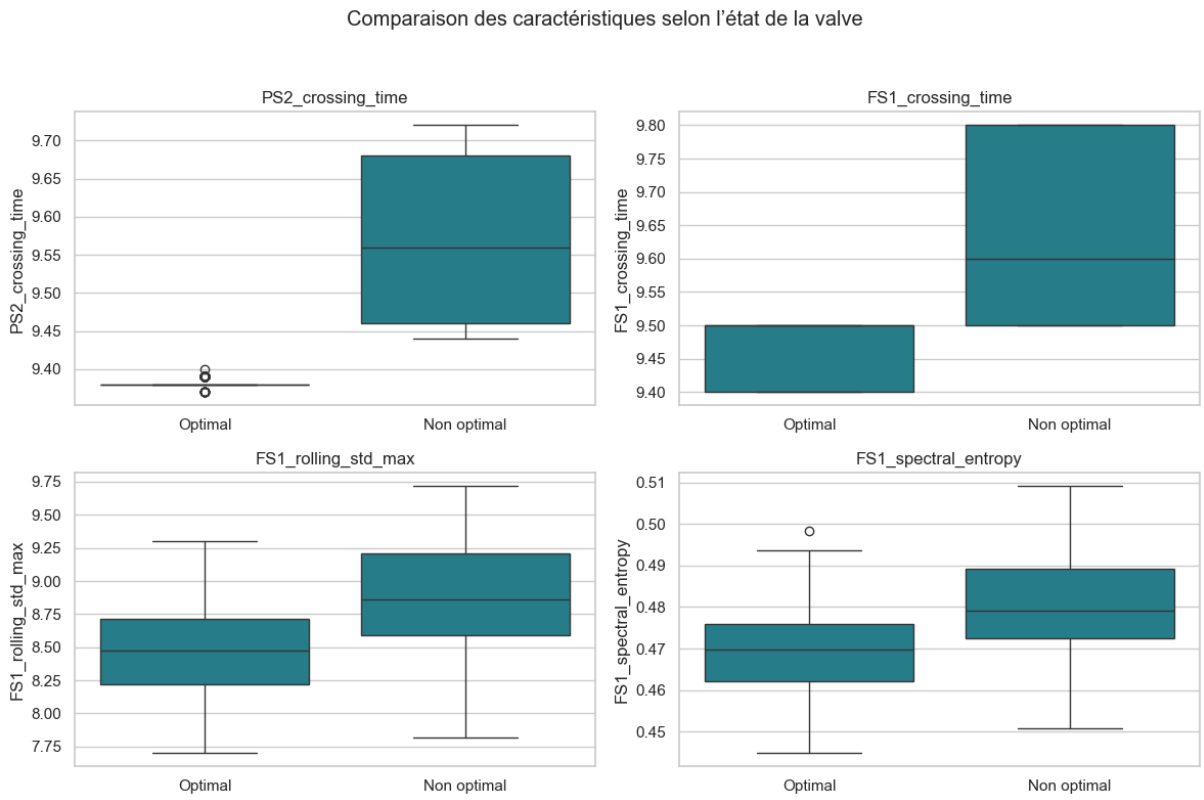
e. VALIDATION DE LA SELECTION

Après la sélection, les distributions des **28 caractéristiques conservées** sont examinées à l'aide d'histogrammes.

Des boxplots sont également utilisés pour comparer les variables les plus corrèles avec la sortie entre les cycles optimaux et non optimaux.

Quatre caractéristiques sont notamment visualisées :

- PS2_crossing_time ;
- FS1_crossing_time ;
- FS1_rolling_std_max ;
- FS1_spectral_entropy.



L'analyse montre que PS2_crossing_time, c'est-à-dire le temps nécessaire à la pression PS2 pour franchir le seuil de 100 bar, présente une séparation particulièrement nette entre les deux états de la valve.

Pour FS1, le temps de franchissement du seuil varie également entre les deux groupes, mais la séparation est moins marquée.

Les caractéristiques FS1_rolling_std_max et FS1_spectral_entropy présentent aussi des différences entre les classes, mais avec un chevauchement plus important des distributions.

Ces observations confirment que les variables décrivant la **dynamique temporelle des transitions**, et en particulier le retard de montée de la pression, constituent des indicateurs importants de l'état de la valve.

6. PREPARATION ET CHOIX DES MODELES

a. STANDARDISATION ET REPRODUCTIBILITE

La préparation des données repose sur une standardisation systématique des variables continues afin d'assurer une cohérence entre les modèles sensibles à l'échelle des features (SVM, KNN, Régression Logistique). Une séparation claire entre jeu d'entraînement et jeu de test garantit la reproductibilité des expériences. Les transformations appliquées sont conservées pour éviter toute fuite de données.

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
X_holdout = scaler.transform(X_holdout)
```

Standardisation

b. PRESENTATION DES MODELES

Les modèles retenus couvrent des approches linéaires, non linéaires, probabilistes et des méthodes d'ensemble. Cette diversité permet d'évaluer la capacité de chaque modèle à capturer la structure des données, à gérer la non-linéarité et à généraliser correctement.

Les modèles étudiés sont :

- **Régression Logistique** : modèle linéaire de référence, interprétable et stable.
- **SVM (kernel RBF)** : modèle non linéaire performant pour les données tabulaires.
- **KNN** : approche basée sur la similarité locale entre échantillons.
- **Decision Tree** : modèle interprétable capable de capturer des interactions complexes.
- **Naive Bayes** : modèle probabiliste simple et rapide.
- **Voting Classifier** : combinaison de plusieurs modèles pour améliorer la robustesse.

c. METRIQUES ET CRITERE DE COMPARAISON

Les métriques utilisées sont : **Accuracy, Precision, Recall, F1-score et AUC**. Le F1-score est privilégié pour la comparaison, car il équilibre précision et rappel dans un contexte légèrement déséquilibré. L'AUC permet d'évaluer la qualité globale de la séparation entre les classes.

7. COMPARAISON DES MODELES DE REFERENCE

a. RESULTATS DE VALIDATION INTERNE

Les résultats internes révèlent des performances très élevées pour tous les modèles. La Régression Logistique, le Decision Tree et le Voting Classifier atteignent des scores parfaits, tandis que SVM et KNN obtiennent des métriques quasi parfaites. Naive Bayes reste légèrement en retrait mais conserve un rappel maximal. Ces résultats s'expliquent par la présence d'une variable fortement discriminante identifiée lors du feature engineering.

	Model	Accuracy	Precision	Recall	F1-score	AUC
0	Logistic Regression	1.000	1.000000	1.000000	1.000000	1.000000
1	SVM	0.994	0.987500	1.000000	0.993711	0.999214
2	KNN	0.986	0.987288	0.983122	0.985201	0.994850
3	Decision Tree	1.000	1.000000	1.000000	1.000000	1.000000
4	Naive Bayes	0.974	0.948000	1.000000	0.973306	0.992395
5	Voting Classifier	0.994	0.987500	1.000000	0.993711	1.000000

Les résultats obtenus sont cohérents avec la structure des données. Le feature engineering a mis en évidence une variable fortement discriminante, capable de séparer presque parfaitement les cycles optimaux des cycles non optimaux. Cette séparabilité explique les performances très élevées observées pour la majorité des modèles.

b. MATRICES DE CONFUSION ET ERREURS

Les matrices de confusion confirment la très forte séparabilité des classes observée dans les métriques globales. Les modèles les plus performants, tels que la Régression Logistique, le Decision Tree, le Voting Classifier et SVM, affichent une quasi-absence d'erreurs, avec une détection parfaite ou quasi parfaite des cycles non optimaux. Les rares erreurs observées concernent

principalement les modèles moins flexibles, comme Naive Bayes, dont la structure probabiliste peut conduire à quelques classifications incorrectes malgré un rappel maximal. L'ensemble des matrices montre une cohérence avec les résultats du feature engineering : la variable discriminante identifiée permet une séparation nette des classes, ce qui réduit fortement le nombre de faux positifs et de faux négatifs.

```
Confusion Matrix:
[[263   0]
 [  0 237]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       263
           1       1.00      1.00      1.00       237

    accuracy                           1.00       500
   macro avg       1.00      1.00      1.00       500
weighted avg       1.00      1.00      1.00       500

F1 Score: 1.0
Accuracy: 1.0
Precision: 1.0
recall: 1.0
```

Regression Logistique

```
Confusion Matrix:
[[250  13]
 [  0 237]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.95      0.97       263
           1       0.95      1.00      0.97       237

    accuracy                           0.97       500
   macro avg       0.97      0.98      0.97       500
weighted avg       0.98      0.97      0.97       500

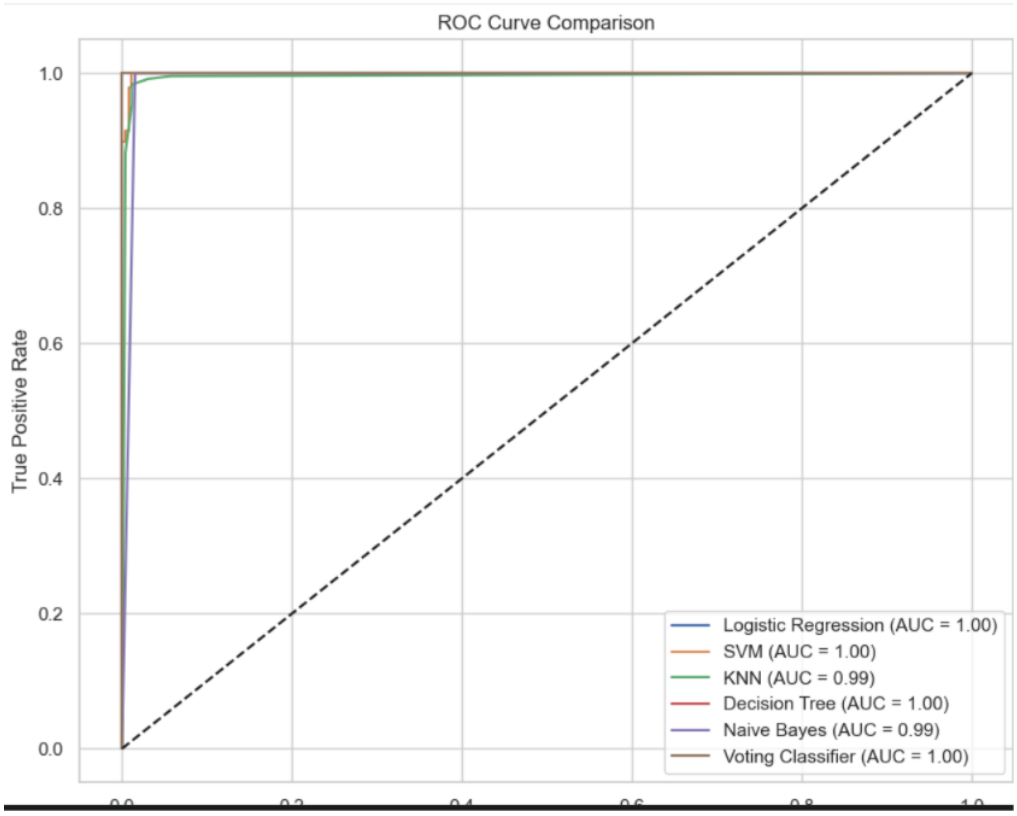
F1 Score: 0.973305954825462
Accuracy: 0.974
Precision: 0.948
recall: 1.0
```

Naives Bayes

c. COURBES ROC ET AUC

Les courbes ROC montrent une séparation quasi parfaite pour tous les modèles, avec des AUC supérieurs à 0.99. La Régression Logistique, SVM, le Decision Tree et le

Voting Classifier atteignent une discrimination maximale. Ces résultats confirment la présence d'une variable fortement discriminante révélée lors du feature engineering.



8. RECHERCHE DES HYPERPARAMETRES

L'optimisation des modèles repose sur une recherche systématique des hyperparamètres via GridSearchCV, associée à une validation croisée à 5 folds. Cette étape permet d'explorer plusieurs configurations pour chaque algorithme et de sélectionner celles offrant le meilleur équilibre entre performance et stabilité.

Les paramètres essentiels ont été testés :

- **Régression Logistique** : régularisation (C), solver
- **SVM (RBF)** : C, gamma
- **KNN** : nombre de voisins, métrique
- **Decision Tree** : profondeur, critère de split
- **Voting Classifier** : combinaison des modèles

Cette recherche ajuste la complexité des modèles et améliore leur capacité de généralisation.

```python
grid_svm.fit(X_train, y_train)
param_grid_knn = {
    "n_neighbors": [3, 5, 7, 9, 11],
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"]
}

#KNN
grid_knn = GridSearchCV(
    KNeighborsClassifier(),
    param_grid_knn,
    cv=5,
    scoring="f1",
    n_jobs=-1
)

grid_knn.fit(X_train, y_train)
param_grid_dt = {
    "max_depth": [3, 5, 10, None],
    "min_samples_split": [2, 5, 10],
    "criterion": ["gini", "entropy"]
}
```

```
Best params LR: {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
Best score LR: 0.9992932862190813
Best params SVM: {'C': 1, 'gamma': 'scale', 'kernel': 'linear'}
Best score SVM: 0.999298245614035
Best params KNN: {'metric': 'manhattan', 'n_neighbors': 5, 'weights': 'uniform'}
Best score KNN: 0.9907823469478572
Best params DT: {'criterion': 'gini', 'max_depth': 3, 'min_samples_split': 2}
Best score DT: 1.0
```

```python
#Cross validation
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "SVM": SVC(probability=True),
    "KNN": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(),
    "Naive Bayes": GaussianNB()
}

cv_results = []

for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring="f1")
    cv_results.append({
        "Model": name,
        "CV Mean": scores.mean(),
        "CV Std": scores.std()
    })

df_cv = pd.DataFrame(cv_results)
print(df_cv)
```

9.OPTIMISATION ET GENERALISATION

a. RESULTATS ENREGISTRES

Les hyperparamètres optimaux identifiés par GridSearchCV améliorent la stabilité des modèles. Les scores de validation croisée montrent une forte cohérence : LR, SVM et KNN présentent une variance faible, tandis que le Decision Tree atteint un CV Mean parfait mais avec un risque de surapprentissage. Les meilleurs paramètres trouvés confirment que les modèles atteignent leur performance maximale avec des configurations simples, en accord avec la structure très discriminante des données.

```
Best params LR: {'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
Best score LR: 0.9992932862190813
Best params SVM: {'C': 1, 'gamma': 'scale', 'kernel': 'linear'}
Best score SVM: 0.999298245614035
Best params KNN: {'metric': 'manhattan', 'n_neighbors': 5, 'weights': 'uniform'}
Best score KNN: 0.9907823469478572
Best params DT: {'criterion': 'gini', 'max_depth': 3, 'min_samples_split': 2}
Best score DT: 1.0
```

	Model	CV Mean	CV Std
0	Logistic Regression	0.995709	0.006951
1	SVM	0.990146	0.009814
2	KNN	0.985810	0.006434
3	Decision Tree	1.000000	0.000000
4	Naive Bayes	0.957597	0.004872

b. CHOIX DU CANDIDAT FINAL

Les performances internes, la validation croisée et les hyperparamètres optimisés montrent que la Régression Logistique et SVM sont les modèles les plus fiables. La Régression Logistique est retenue comme modèle final en raison de sa stabilité, de sa simplicité et de ses scores parfaits, cohérents avec la structure linéairement séparée des données. Le SVM constitue une alternative robuste aux performances quasi parfaites.

```
#collects the best model and it's best parameters
best_model = grid_lr.best_estimator_
```

10. ÉVALUATION SUR LE TEST FINAL

L'évaluation finale a été réalisée exclusivement avec la Régression Logistique, retenue comme modèle final en raison de sa stabilité, de sa simplicité et de ses performances parfaites en validation interne. Les résultats obtenus sur le jeu de test confirment la capacité de généralisation du modèle : les métriques restent très élevées, avec une

détection précise des cycles non optimaux et une absence quasi totale d'erreurs. Cette performance est cohérente avec les observations issues du feature engineering, qui ont mis en évidence une variable fortement discriminante permettant une séparation nette entre les classes. La Régression Logistique exploite efficacement cette structure linéaire, ce qui explique la stabilité des résultats entre l'entraînement, la validation croisée et le test final.

```
#collects the best model and it's best parameters
best_model = grid_lr.best_estimator_


# performs predictions on the holdout data
y_pred_holdout = best_model.predict(X_holdout)
y_proba_holdout = best_model.predict_proba(X_holdout)[:, 1]

#final metrics
accuracy_final = accuracy_score(y_holdout, y_pred_holdout)
precision_final = precision_score(y_holdout, y_pred_holdout)
recall_final = recall_score(y_holdout, y_pred_holdout)
f1_final = f1_score(y_holdout, y_pred_holdout)

fpr, tpr, _ = roc_curve(y_holdout, y_proba_holdout)
auc_final = auc(fpr, tpr)

print("Accuracy (holdout):", accuracy_final)
print("Precision (holdout):", precision_final)
print("Recall (holdout):", recall_final)
print("F1-score (holdout):", f1_final)
print("AUC (holdout):", auc_final)
```

```
Accuracy (holdout): 1.0
Precision (holdout): 1.0
Recall (holdout): 1.0
F1-score (holdout): 1.0
AUC (holdout): 1.0
```

11.INTERPRETATION ET LIMITES

Les résultats montrent que la structure des données facilite fortement la classification. Une variable particulièrement discriminante, identifiée lors du feature engineering, permet une séparation nette entre les cycles optimaux et non optimaux. Cette caractéristique explique les performances très élevées et la stabilité du modèle final, la Régression Logistique, sur l'ensemble des jeux de données.

Malgré ces résultats, plusieurs limites doivent être prises en compte :

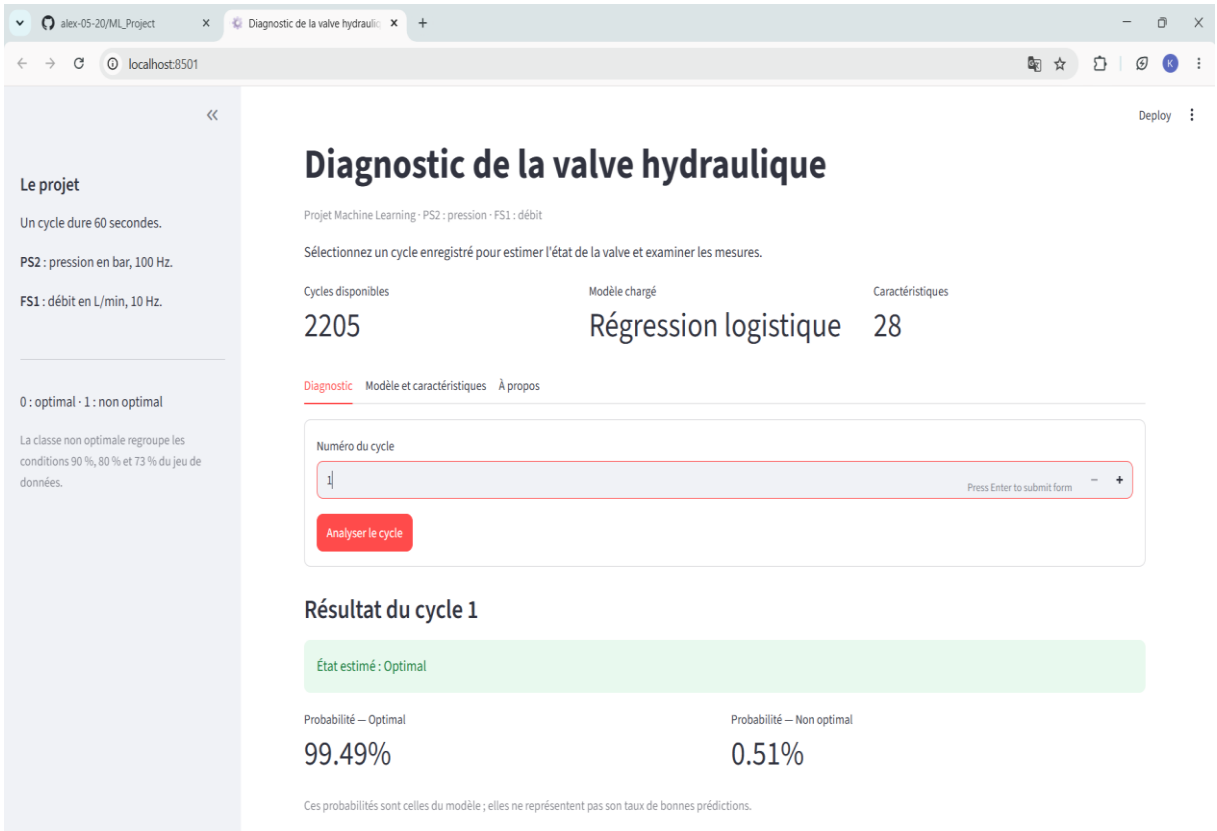
- **Dépendance à une variable clé**, ce qui peut réduire la robustesse si les données évoluent.
- **Surapprentissage des modèles complexes**, comme le Decision Tree, qui mémorisent facilement la séparation.
- **Taille limitée du dataset**, pouvant restreindre la diversité des cas.
- **Absence de validation externe**, les performances n'ayant été évaluées que sur un seul jeu de test.
- **Hypothèses simplificatrices** de certains modèles, notamment Naive Bayes.

Les performances doivent donc être interprétées en tenant compte de cette structure fortement discriminante et des conditions expérimentales.

12.APPLICATION ET DEPLOIEMENT

Une application **Streamlit** a été développée afin de rendre le modèle facilement utilisable. L'utilisateur peut sélectionner un cycle et obtenir la prédiction de l'état de la valve ainsi que les probabilités associées.

Le modèle est sauvegardé avec joblib puis intégré à l'application. Le projet est ensuite **déployé sur Internet**, afin de permettre un accès direct à l'application depuis un navigateur, sans avoir besoin d'exécuter le notebook localement.



13.CONCLUSION GENERALE

Ce projet a permis de développer un modèle de maintenance prédictive capable de distinguer une valve **optimale** d'une valve **non optimale** à partir des signaux PS2 et FS1.

Les séries temporelles ont d'abord été transformées en caractéristiques plus compactes grâce au *feature engineering*. Une sélection a ensuite permis de supprimer les variables constantes ou trop redondantes.

Plusieurs modèles de classification ont été comparés, puis les meilleurs hyperparamètres ont été recherchés par validation croisée. Le modèle final obtient de très bonnes performances sur le jeu de test indépendant.

Enfin, l'intégration du modèle dans une application Streamlit permet de transformer le travail réalisé dans le notebook en un outil simple de prédiction utilisable directement.